



Ministry of Higher Education and Scientific Research of Tunisia
Higher Institute of Management

Graduation Project Report

Bachelor's Degree in Business Intelligence

A Decision Making Solution for the Production Activity of Maretap

Host Company



Elaborated by

Gorrab Mohamed Aziz
Ben Nejma Ahmed Abderrahmen

Academic Supervisor:
Mrs. Siwar Gorrab

Professional Supervisor:
Mr. Nourdine

2025–2026

Contents

List of Acronyms	v
1 AI Powered Chatbot & Document Intelligence	1
1.1 Introduction	1
1.2 Sprint 3 User Stories: AI Chatbot Module	1
1.2.1 RAG Chatbot and AI Powered Analysis Functional Analysis	2
1.3 Requirements Analysis	3
1.3.1 Functional Requirements	3
1.3.2 Non Functional Requirements	4
1.4 RAG Architecture: Design Choices	5
1.4.1 Technology Stack	5
1.5 RAG Pipeline Implementation	6
1.5.1 Ingestion Phase	6
1.5.1.1 Step 1: Text Extraction	6
1.5.1.2 Step 2: Chunking	7
1.5.1.3 Step 3: Embedding	8
1.5.1.4 Step 4: Indexing	8
1.5.2 Query Phase	8
1.5.2.1 Step 1: Query Embedding	8
1.5.2.2 Step 2: Semantic Retrieval	8
1.5.2.3 Step 3: Context Assembly	8
1.5.2.4 Step 4: Task Classification and Prompt Selection	9
1.5.2.5 Step 5: Response Generation	10
1.5.2.6 Step 6: Post-Processing	10
1.6 Session Management	11
1.6.1 Conversation Sessions	11
1.6.2 Conversation History as LLM Context	11
1.6.3 Morning Suggestions	12
1.7 Collaborative Features	13
1.7.1 Response Rating and Collaborative Annotation	13
1.7.2 Session Sharing	14

1.8 Document Upload in Chatbot Context	15
1.9 Charts in Chatbot Responses	15
1.10 User Memory	16
1.11 Admin Statistics View	18
1.12 Conclusion	18

List of Figures

1.1	Use Case Diagram for the RAG Chatbot & AI Powered Analysis	3
1.2	Text splitter configuration [?]	7
1.3	RAG Pipeline Architecture (Ingestion and Query Phases)	10
1.4	Conversation history injection into the LLM prompt	12
1.5	Chatbot Morning Suggestions at Session Start	13
1.6	Collaborative comment and rating controls on a chatbot response	13
1.7	Read only shared session	14
1.8	Document attachment and library pre selection in the chatbot interface . .	15
1.9	Chatbot response with inline Chart.js production trend	16
1.10	User Memory Update Logic in the Chatbot Service	17
1.11	Chatbot Admin Statistics Dashboard	18

List of Tables

1.1	Sprint 3: AI Chatbot Advanced Features User Stories	2
1.2	RAG Technology Stack: EZZAOUIA Chatbot	5
1.3	Specialized Prompt Templates and Their Task Scope	9

List of Acronyms

AI Artificial Intelligence

API Application Programming Interface

BI Business Intelligence

BOPD Barrels Of Petroleum per Day

BSW Basic Sediment and Water

BWPD Barrels of Water Per Day

CPU Central Processing Unit

CSRF Cross Site Request Forgery

DWH Data Warehouse

ETL Extract Transform Load

GOR Gas Oil Ratio

IT Information Technology

JDBC Java DataBase Connectivity

KPI Key Performance Indicator

LLM Large Language Model

MSCF Million Standard Cubic Feet

ORM Object Relational Mapping

RAG Retrieval Augmented Generation

RBAC Role Based Access Control

REST Representational State Transfer

SMTP Simple Mail Transfer Protocol

SPA Single Page Application

STBD Stock Tank Barrels per Day

TTL Time To Live

WSGI Web Server Gateway Interface

Chapter 1

AI Powered Chatbot & Document Intelligence

1.1 Introduction

The AI powered chatbot represents the most technically advanced component of the EZZAOUIA platform. It allows MARETAP S.A. engineers to query production records and technical documents using natural language instead of manually searching through reports and spreadsheets. This chapter introduces and describes the Retrieval Augmented Generation (**R**etrieval **A**ugmented **G**eneration (RAG)) [?] architecture that powers the module. RAG combines semantic document retrieval from a ChromaDB [?] vector store with a locally hosted Large Language Model (**L**arge **L**anguage **M**odel (LLM)) via Ollama [?], ensuring that every response is grounded in company specific data and that no information leaves the MARETAP internal network.

The following sections detail the full RAG pipeline from document ingestion to response generation, the session management system, and the collaborative features including response rating, public annotations, and session sharing that transform individual queries into a shared knowledge resource for the production team.

1.2 Sprint 3 User Stories: AI Chatbot Module

Period: (April 3, May 2, 2026)

The following user stories detail the advanced features of the RAG chatbot introduced in US,26, covering session management, document upload, collaborative annotation, and administrator statistics.

Table 1.1: Sprint 3: AI Chatbot Advanced Features User Stories

ID	Role	User Story
US,32	User	As a user, I want to start a new chat session and ask production related questions in natural language, and stop an ongoing generation at any time, so that I can get instant answers without writing SQL queries.
US,33	User	As a user, I want to upload documents directly in the chatbot and receive intelligent morning suggestions and follow up proposals after each answer so that the AI has richer context and guides my exploration.
US,34	User	As a user, I want to annotate, and share chatbot sessions with colleagues so that the team can collaborate around AI generated insights.
US,35	Admin	As an admin, I want to view global chatbot usage statistics so that I can monitor adoption and answer quality across the team.

1.2.1 RAG Chatbot and AI Powered Analysis Functional Analysis

The EZZAOUIA platform features an AI powered assistant for decision support using natural language processing and a hybrid Retrieval Augmented Generation pipeline [?]. As illustrated in Figure 4.5, users interrogate indexed documentation via a specialized backend pipeline. Upon a query, the system combines two context sources: semantic retrieval from a ChromaDB [?] vector database and live production data fetched directly from the SQL Server data warehouse (field KPIs, well rankings, tank levels, monthly trends). The enriched context is then submitted to a locally hosted LLM via Ollama [?], and each response is accompanied by follow up suggestions generated by a dedicated rule-based function. The assistant supports dynamic charts rendered from live well production data queried directly from the SQL warehouse. Documents uploaded through the ingestion pipeline and processed asynchronously via Celery [?] become available to the knowledge base and can be pre selected to focus the retrieval context. A collaborative ecosystem allows users to share sessions via public token or direct user to user sharing, provide satisfaction ratings, and engage in technical comments anchored to individual responses. The Administrator maintains exclusive access to usage statistics to monitor performance within MARETAP S.A.



Figure 1.1: Use Case Diagram for the RAG Chatbot & AI Powered Analysis

Figure 1.1 presents the use case diagram for the RAG chatbot and AI powered analysis module. It maps the interactions between the two system roles and the available features: users can initiate chat sessions, upload documents, rate responses, annotate answers, and share sessions with colleagues, while administrators additionally access the global usage statistics dashboard.

1.3 Requirements Analysis

1.3.1 Functional Requirements

- The system shall provide a natural language conversational interface supporting French, English, and Arabic, grounded in the company own documents and production data.
- The system shall implement a RAG pipeline [?] that embeds questions, retrieves relevant document chunks from ChromaDB [?], and generates answers using a local

LLM, ensuring no data leaves the MARETAP internal network.

- The system shall persist full conversation history server side and allow users to create, rename, delete, and revisit past sessions.
- The system shall allow documents uploaded directly in the chat interface to be indexed and used immediately as context for subsequent questions.
- The system shall support querying multiple document collections simultaneously by accepting a list of document identifiers at inference time.
- The system shall allow users to rate responses and attach public or private comments to any answer.
- The system shall allow users to share sessions with specific colleagues, accessible through a dedicated shared inbox.
- The system shall generate contextual follow up question suggestions after each response using a rule-based template system, and produce morning suggestions refreshed every four hours via in process cache.
- The system shall generate Chart.js visualisations alongside textual answers when a quantitative question is detected, by querying the SQL warehouse independently of the LLM.
- The system shall expose a stop mechanism allowing users to cancel ongoing LLM inference at any point.
- Chatbot usage statistics shall be restricted to administrators only.
- The system shall maintain a persistent memory model per user, storing summaries organised by well and topic to personalise future responses.

1.3.2 Non Functional Requirements

- **Privacy:** All LLM inference is performed locally via Ollama llama3.1:8b [?]. No data is sent to any external cloud API.
- **Performance:** Document embedding runs asynchronously via Celery [?]. Morning suggestions are cached in process (Django LocMemCache) for four hours to avoid regenerating them on every page load.
- **Accuracy:** The RAG approach [?] grounds every response in retrieved source material, making answers auditable via cited chunks.

- **Scalability:** ChromaDB [?] collections are isolated per document identifier, allowing new documents to be added without affecting existing indexes.
- **Security:** All chatbot endpoints require an active Django [?] session. Private comments are filtered server side based on the authenticated user identity.

1.4 RAG Architecture: Design Choices

Why RAG?

The choice of Retrieval Augmented Generation (RAG) is based on its ability to optimize the output of Large Language Models by referencing a specific knowledge base before generating a response [?]. Two primary methods ground language models in domain knowledge: fine tuning and RAG. Fine tuning requires intensive GPU resources and constant retraining as new data is added. RAG was selected for the platform because it retrieves relevant document chunks at query time to provide context [?]. This approach offers three key advantages:

- **No Training Cost:** New documents are available instantly through the ingestion pipeline without model retraining.
- **Auditability:** Every response is traceable to specific source chunks, ensuring all answers are verifiable.
- **On Premise Security:** The system runs entirely on internal MARETAP servers via Ollama [?] to maintain total data privacy.

1.4.1 Technology Stack

Table 1.2: RAG Technology Stack: EZZAOUIA Chatbot

Component	Technology	Role
LLM	Ollama llama3.1:8b [?] (CPU) via LangChain	A Large Language Model (LLM) is an advanced deep learning algorithm trained to recognize, summarize, and generate content with human like proficiency [?].
Embeddings	SentenceTransformer all-MiniLM-L6-v2 [?]	CPU based dense vector encoding of documents and queries.

Table 1.2 (continued)

Component	Technology	Role
Vector Store	ChromaDB [?]	A specialized database that manages information as high dimensional mathematical representations (vectors) to facilitate efficient similarity searching based on semantic meaning [?].
Text Splitter	RecursiveCharacterTextSplitter [?]	A semantic chunking utility that recursively splits documents into smaller pieces based on a list of characters to maintain contextual integrity [?].
PDF Extraction	PyPDF2 [?]	Page by page text extraction from PDF documents.
DOCX Extraction	python-docx [?]	Extraction from Word paragraphs, tables, and headers.
XLSX Extraction	pandas [?]	Row level extraction from the first sheet of the workbook. XLSX files are parsed for structured data but are not indexed into the vector store.
Async Processing	Celery [?] + Redis [?]	Background ingestion pipeline with status tracking.

1.5 RAG Pipeline Implementation

The RAG pipeline operates in two independent phases: the **ingestion phase** runs asynchronously when documents are uploaded, and the **query phase** runs synchronously when a user submits a question.

1.5.1 Ingestion Phase

1.5.1.1 Step 1: Text Extraction

The uploaded file is parsed according to its format:

- PDF: PyPDF2 [?] extracts text page by page, handling multi column layouts common in TCM and intervention reports.
- DOCX: python-docx [?] iterates over paragraphs and table cells, extracting all textual content including headers and footnotes.

- **XLSX:** pandas [?] reads the first sheet of the workbook, extracting row level structured data. XLSX files are processed for structured data extraction but are not chunked or embedded into ChromaDB; they therefore cannot be queried through the RAG retrieval pipeline.

Before indexing, the pipeline performs a petroleum domain validation: if the document does not contain content related to petroleum operations or MARETAP, its status is set to rejected and the file is physically deleted. This prevents irrelevant documents from polluting the knowledge base.

1.5.1.2 Step 2: Chunking

The extracted text of PDF and DOCX files is divided into overlapping chunks using RecursiveCharacterTextSplitter [?]:

- **Chunk size:** 1,200 characters
- **Overlap:** 500 characters between consecutive chunks

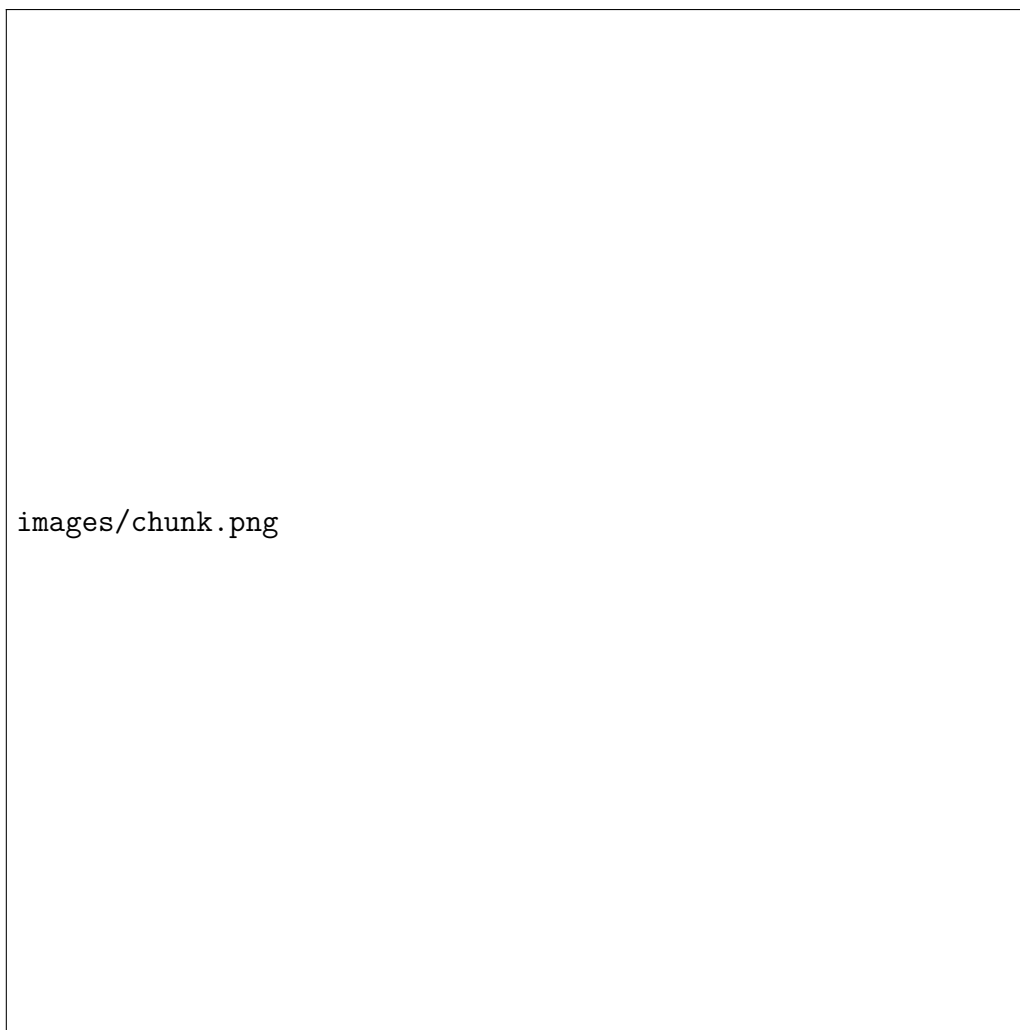


Figure 1.2: Text splitter configuration [?]

Figure 1.2 shows the Python code used to configure the `RecursiveCharacterTextSplitter` during the ingestion phase. The snippet defines a chunk size of 1,200 characters, an overlap of 500 characters between consecutive segments, and an ordered list of separators (double newline, single newline, period, and space) that the splitter uses to find natural break points before falling back to character-level splitting.

The overlap ensures that semantic continuity is preserved across chunk boundaries: a sentence split across two chunks appears in both, preventing context loss during retrieval.

1.5.1.3 Step 3: Embedding

Each chunk is converted into a dense vector using SentenceTransformer all-MiniLM-L6-v2 [?], which runs entirely on CPU. The same model is used for both document embedding and query embedding, ensuring vector space consistency.

1.5.1.4 Step 4: Indexing

Vectors are inserted into ChromaDB [?] with source metadata including `filename`, `file_type`, `uploaded_by`, and `doc_id`. The collection name is derived from a hash of the document identifier to guarantee uniqueness. Failed ingestion tasks are automatically retried up to three times with a 60 second delay between attempts.

1.5.2 Query Phase

1.5.2.1 Step 1: Query Embedding

The user question is embedded using the same SentenceTransformer model [?], producing a query vector in the same semantic space as the stored chunk embeddings.

1.5.2.2 Step 2: Semantic Retrieval

ChromaDB [?] performs a nearest neighbour search over the stored embeddings. The pipeline retrieves between 6 and 20 chunks depending on question type, defaulting to 6 for production KPI queries, 8 for document-priority questions, 10 for explicit document-only questions, and up to 20 when the user specifies `from docs` without attaching a file. The pipeline supports multi-document contexts: a list of document identifiers queries several ChromaDB collections simultaneously.

1.5.2.3 Step 3: Context Assembly

Once the most relevant chunks have been retrieved, they are concatenated into a single structured context block before being passed to the LLM. Each chunk is prefixed with its

source attribution, indicating the originating document and its position within it, allowing the model to ground its response in traceable references.

1.5.2.4 Step 4: Task Classification and Prompt Selection

Before invoking the LLM, the pipeline runs `_detect_task()`, which classifies the incoming question into one of five task types and selects the corresponding prompt template:

Table 1.3: Specialized Prompt Templates and Their Task Scope

Prompt	Scope and Constraints
PROMPT_PRODUCTION_KPI	Production KPI questions (top producers, BSW, GOR, rankings). Uses SQL context only; document context is excluded. Enforces exact ranking order from the SQL result set.
PROMPT_DOCUMENT_QA	Document question-answering (workover history, abandonment reasons, completion design). Uses retrieved chunks only; SQL data is ignored. The model is instructed to cite the source document for every factual claim.
PROMPT_WELL_ANALYSIS	Combined SQL and document well analysis triggered by keywords such as <i>analyse</i> , <i>tell me about</i> , or <i>from both</i> . Structures the answer into a Production Performance table (SQL) and an Operational History table (documents).
PROMPT_OPERATIONAL_EVENTS	Field operational events extracted from the DimDate comments column of the Data Warehouse. Used for questions about daily activities, monitoring periods, and shut-in events.
PROMPT_FIELD_SUMMARY	Field-level KPI summary using SQL aggregates. Structures the answer with an executive summary, a KPI table with benchmarks, and a critical alerts section for wells with BSW above 80%.

This specialisation allows the system to behave correctly across very different question types using the same underlying model: `PROMPT_PRODUCTION_KPI` forbids the model from using document context, while `PROMPT_DOCUMENT_QA` forbids it from inventing SQL data.

1.5.2.5 Step 5: Response Generation

The context block, the conversation history, and the user question are assembled into a prompt and submitted to Ollama [? ?] using the selected task prompt. The prompt instructs the Dr. EZZAOUIA persona to answer exclusively from the provided context, structure responses according to the detected task type, cite source documents for factual claims, and respond in the same language as the question.

1.5.2.6 Step 6: Post-Processing

After the LLM returns its response, two independent post-processing steps run without involving the model:

- **Follow-up suggestions:** the `generate_suggestions()` function produces three contextual follow-up questions from a rule-based template system parameterised by the detected well code and the question type. These are never generated by the LLM.
- **Chart data:** if `detect_chart_request()` identifies both a chart keyword and a recognised well code in the question, `build_chart_data()` queries the SQL warehouse directly to construct a Chart.js [?] compatible JSON object containing two series: monthly oil production and BSW percentage. The LLM produces no JSON; the chart is assembled from live warehouse data and attached to the response independently.

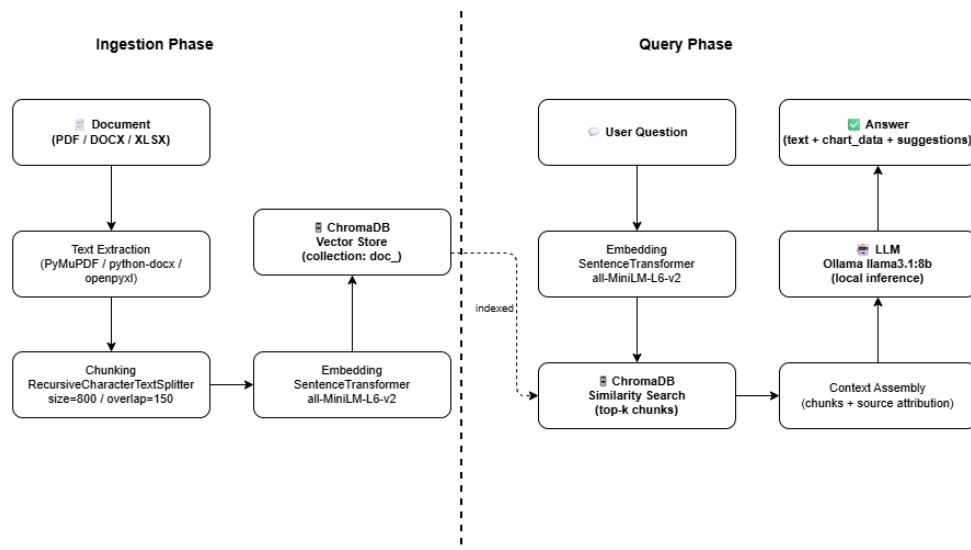


Figure 1.3: RAG Pipeline Architecture (Ingestion and Query Phases)

Figure 1.3 provides a full architectural view of the RAG pipeline across its two phases. The ingestion phase on the left shows the sequential flow from file upload through text

extraction, domain validation, chunking, embedding, and storage into ChromaDB. The query phase on the right illustrates how a user question is embedded, matched against stored vectors, classified by `_detect_task()`, and combined with the retrieved context and conversation history before being submitted to the local LLM. Follow-up suggestions and chart data are generated after the LLM call by independent post-processing functions.

1.6 Session Management

1.6.1 Conversation Sessions

Each chat session is a server side entity grouping a sequence of messages under a unique identifier, stored in the Django [?] database and associated with the authenticated user.

1.6.2 Conversation History as LLM Context

When a user submits a question within an existing session, the most recent exchange is included in the prompt as conversation history. Only the last question and the first 200 characters of its answer are injected, allowing the LLM to resolve immediate follow up references. This keeps the prompt concise while maintaining basic conversational continuity between two consecutive turns. History injection is conditional: it is only applied when the same well code appears in both the previous and the current question, preventing cross-well context contamination when the user switches wells or asks a general field question.



Figure 1.4: Conversation history injection into the LLM prompt

Figure 1.4 shows the Python code responsible for injecting conversation history into the LLM prompt. The snippet retrieves the last exchange from the session history, checks that both questions reference the same well code, formats the previous question and a 200-character truncation of its answer into a structured **PREVIOUS EXCHANGE** block, and passes the assembled string to the prompt builder.

1.6.3 Morning Suggestions

Morning suggestions are available between 06h00 and 17h00. They are generated on demand from the Data Warehouse and cached in process (Django LocMemCache) with a four hour **Time To Live** (TTL). Suggestions are derived exclusively from production anomalies detected in the DWH including wells with BOPD decline, BSW above 70 percent, or abnormal GOR values.

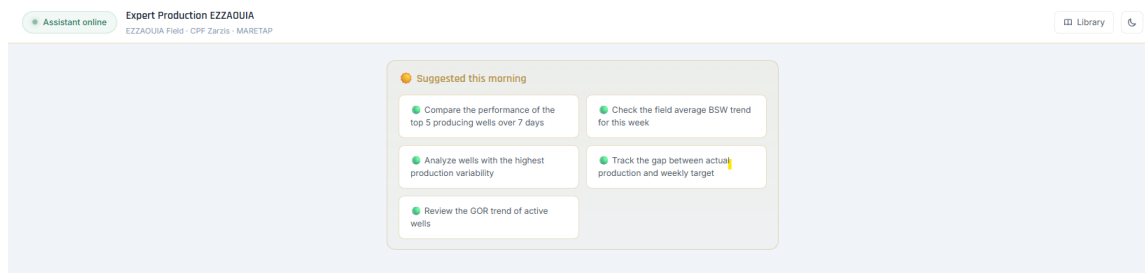


Figure 1.5: Chatbot Morning Suggestions at Session Start

Figure 1.5 shows the morning suggestions panel displayed at the start of a chat session. The interface presents a set of automatically generated questions derived from production anomalies detected in the Data Warehouse, giving engineers a data-driven starting point for their daily analysis without needing to formulate queries manually.

1.7 Collaborative Features

1.7.1 Response Rating and Collaborative Annotation

Each chatbot response features thumbs up and thumbs down controls to record user feedback through a boolean satisfaction flag. The administrative interface aggregates these ratings to monitor satisfaction levels and identify low quality responses. Users can annotate answers with public comments to share technical context with the entire team or use private comments for personal notes. This collaborative framework transforms individual queries into a shared knowledge resource, improving the overall reliability of the intelligent platform.



Figure 1.6: Collaborative comment and rating controls on a chatbot response

Figure 1.6 shows the comment and rating controls attached to a chatbot response. The thumbs up and thumbs down buttons allow users to record their satisfaction, while the comment area supports both public annotations visible to all team members and private notes restricted to the author.

The ask pipeline surfaces relevant public comments from past sessions through automated keyword matching. The system extracts up to six significant words (at least four characters each) from the query and compares them against historical questions to provide MARETAP S.A. engineers with immediate peer insights.

1.7.2 Session Sharing

When an authenticated recipient consults a shared session via its token, the viewed flag is automatically set to true. Unauthenticated visitors who open the same link can read the conversation but do not trigger the flag. In both cases, recipients have read only access to the full conversation.

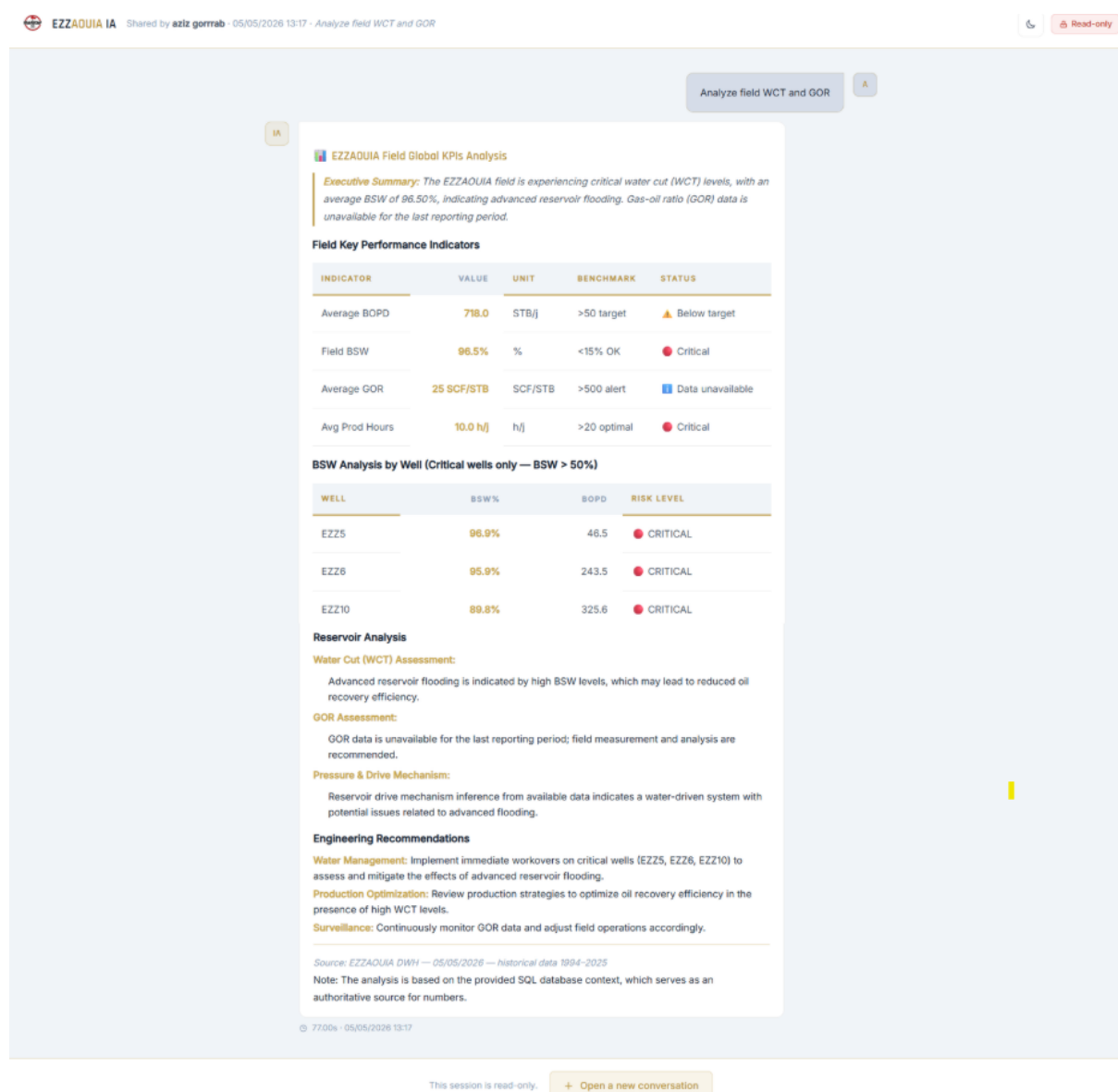


Figure 1.7: Read only shared session

Figure 1.7 illustrates the read-only view of a shared session. The full conversation is

accessible to the recipient through a unique token link, with no ability to send new messages or modify the content.

1.8 Document Upload in Chatbot Context

Unlike the standalone file ingestion module, the chatbot also supports direct document upload within the conversation interface. PDF and DOCX documents are indexed immediately:

1. The file is parsed and chunked on the fly using RecursiveCharacterTextSplitter [?].
2. The resulting chunks are added to ChromaDB [?] with session metadata.
3. Subsequent questions in the same session retrieve context from the uploaded document alongside the general knowledge base.

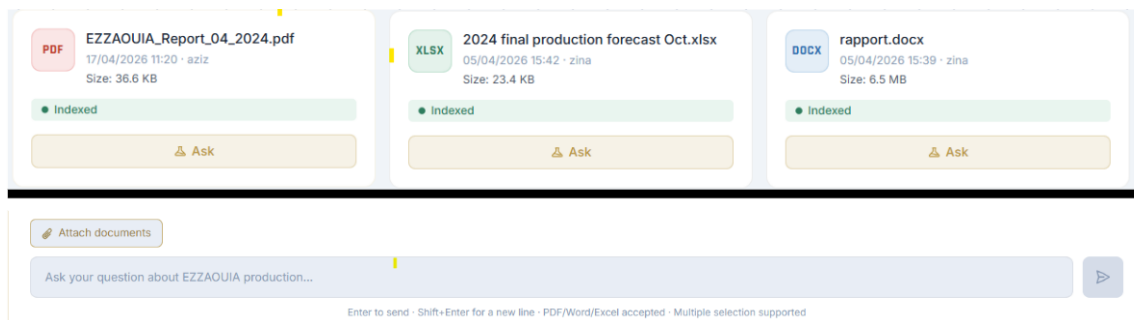


Figure 1.8: Document attachment and library pre selection in the chatbot interface

Figure 1.8 shows the document upload area within the chatbot interface. Users can either attach a new file directly from their workstation or pre-select an existing document from the library to focus the retrieval context on a specific source before asking questions.

It is also possible to pre select a document from the library to initiate a chat session. Supported formats for RAG indexing are PDF and DOCX. XLSX files can be uploaded for structured data viewing but are not indexed into the vector store and therefore cannot be queried through the chatbot.

1.9 Charts in Chatbot Responses

When a user question involves quantitative production data for a specific well, the pipeline detects analytical intent and generates an interactive chart. Detection requires both a chart keyword and a recognised well code in the question. When both conditions are met, the `build_chart_data()` function queries the SQL warehouse directly and constructs a

Chart.js [?] compatible JSON object containing two series: monthly oil production and BSW percentage. This chart assembly step runs entirely outside the LLM, after the model has already returned its textual response.

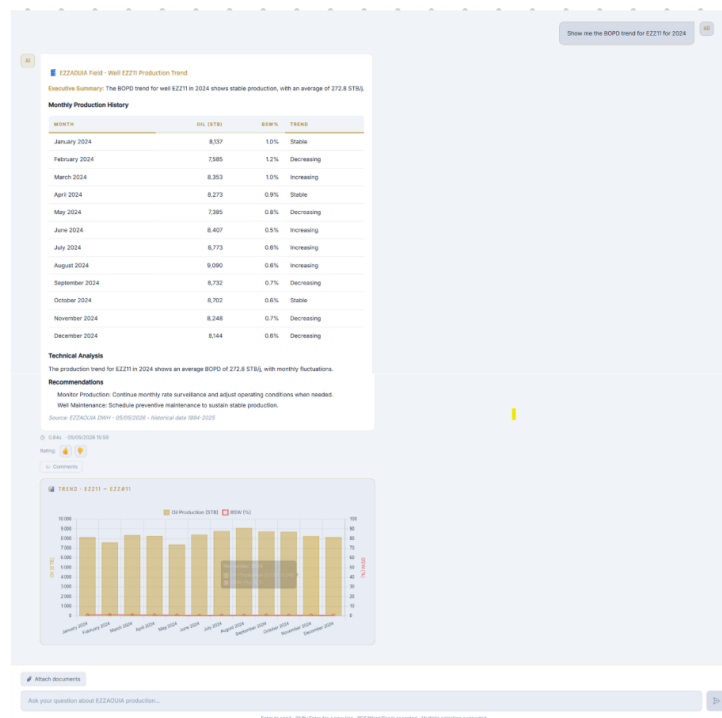


Figure 1.9: Chatbot response with inline Chart.js production trend

Figure 1.9 shows a chatbot response that includes an inline production trend chart. The visualisation is rendered directly in the conversation thread alongside the textual answer, displaying the monthly oil production and BSW series for the queried well so that the engineer can visually inspect historical trends without leaving the chat interface.

The React [?] frontend renders the visualisation inline in the conversation thread. Interactive hover tooltips display both series values simultaneously, allowing engineers to visually inspect monthly production trends and BSW evolution.

1.10 User Memory

User memory provides cross-session persistence by writing one record per user, well, and topic — up to five records per well — across the categories: PRODUCTION, BUDGET, WORKOVER, RESERVOIR, and GENERAL. Each record stores the first 200 characters of the most recent answer for that combination and is overwritten at every subsequent interaction on the same topic.



Figure 1.10: User Memory Update Logic in the Chatbot Service

Figure 1.10 shows the `update_user_memory` Python function that runs at the end of each chatbot interaction. The code detects the topic of the question via `_detect_topic`, extracts the well code from the well object if present, truncates the answer to 200 characters, and then calls `UserMemory.objects.update_or_create` to upsert a memory record keyed on the user, well code, and topic combination.

User memory records are written at the end of each interaction and stored persistently in the database, keyed on user, well, and topic. The retrieval mechanism is implemented: `get_user_memory()` fetches the last ten records and formats them as a structured text block. In the current version, however, this memory context is not yet injected into the LLM prompt, meaning stored summaries do not yet influence response generation. The architecture is designed to support future personalization once the injection is wired into the prompt assembly step.

1.11 Admin Statistics View

The View Chatbot Stats feature is accessible exclusively to admin role users. It provides a consolidated dashboard of chatbot usage metrics:

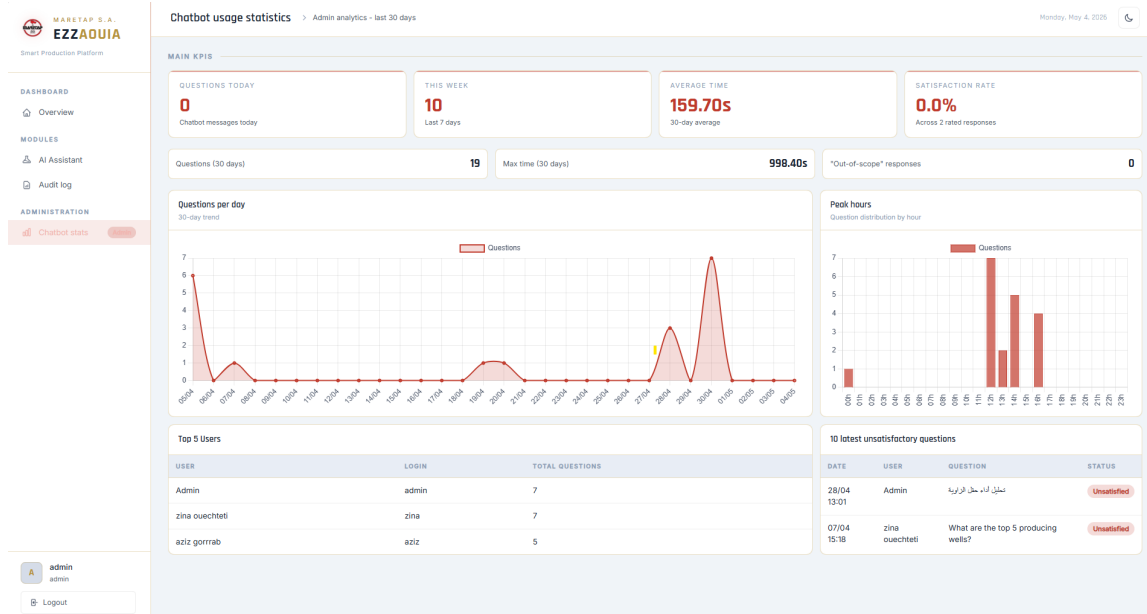


Figure 1.11: Chatbot Admin Statistics Dashboard

Figure 1.11 shows the admin statistics dashboard for the chatbot module. The interface presents usage metrics across several panels including rolling question counts, average and peak response durations, satisfaction rates, out-of-scope query detection, a 30-day daily activity chart, and a peak hour distribution heatmap, giving the administrator a complete operational view of chatbot performance across the team.

The admin statistics dashboard tracks eight metric categories for the MARETAP S.A. platform. Rolling question counts are monitored daily while response durations are tracked over 30 days. The system calculates satisfaction rates and identifies out of scope queries through keyword detection. Visualizations include a 30 day activity chart and a peak hour distribution map.

1.12 Conclusion

This chapter presented the AI powered chatbot module of the EZZAOUIA platform, covering the full RAG pipeline [?] from document ingestion to response generation. The architecture combines a locally hosted LLM via Ollama [?] with ChromaDB [?] vector retrieval and SentenceTransformer embeddings [?] to deliver grounded and auditable answers from MARETAP production data and technical documents. A task classification layer selects one of five specialized prompt templates at query time, ensuring that production KPI questions, document Q&A, well analyses, operational history queries, and field

summaries are each handled with purpose-built instructions. Follow-up suggestions and production charts are generated by independent post-processing functions after the LLM call, keeping the model focused on textual reasoning. Collaborative features (including session sharing, response rating, and public annotations) transform individual queries into a shared knowledge resource for the engineering team. The following chapter introduces the Production Forecasting module, extending the platform from retrospective analysis to predictive insights.